KODEWALA ACADEMY

Core Java

Interview Questions & Answers

Part A: Core Java Fundamentals | Part B: Methods, Control Flow, Arrays & Strings

23 topics • 451 questions • Notes pages 1–103

Java Interview

How to use this document

- Questions follow the same order as your notes. The page range of each topic is written next to its heading.
- **Q** = interview question, **Ans** = short, easy answer. Shaded boxes contain code you can write in the interview.
- Comparison tables (e.g., String vs StringBuilder) are very popular – learn them by heart.
- Many questions come directly from the "Interview Experience" sections of your notes; the rest are common follow-up questions interviewers ask on the same topic.

Contents

#	Topic	Notes Pages	Questions
PART A	**Core Java Fundamentals**		
1	Java Introduction	1–4	23
2	First Java Program	5–8	23
3	JDK, JRE & JVM	9–10	16
4	Command-Line Arguments	11–12	15
5	Identifiers & Naming Conventions	13–17	15
6	Primitive Data Types	18–20	21
7	Non-Primitive Data Types	21–22	14
8	Java Variables	23–27	21
9	Scope of Variables	28–30	15
10	Java Operators	31–33	23
11	User Input — Scanner	34–36	18
PART B	**Methods, Control Flow, Arrays & Strings**		
12	Java Methods	37–43	23
13	Access Modifiers	44–48	16
14	Static vs Instance Methods	49–54	21

#	Topic	Notes Pages	Questions
15	If-Else Statements	55–58	17
16	Loops	59–65	20
17	Switch Statements	66–68	15
18	Break & Continue	69–73	14
19	Java Arrays	74–79	29
20	String Fundamentals	81–90	32
21	String Methods	91–96	26
22	StringBuffer	97–100	17
23	StringBuilder	101–103	17

Kodewala_Java Interview

PART A — Core Java Fundamentals

Pages 1–36

Topic 1: Java Introduction (Pages 1–4)

What is Java, Features, WORA, Platform Independence

Q1. What is Java?

Ans: Java is a high-level, object-oriented, platform-independent programming language. It was created by James Gosling at Sun Microsystems (released in 1995). Oracle bought Sun in 2010 and now maintains Java.

Q2. What was Java's original name?

Ans: Java was first called **Oak**. It was later renamed Java (after Java coffee).

Q3. What are the main features of Java?

- **Platform independent** – same bytecode runs on any OS with a JVM.
- **Object-oriented** – everything is organised in classes and objects.
- **Robust** – strong memory management, exception handling, garbage collection.
- **Secure** – no explicit pointers, bytecode verification, Security Manager/sandbox.
- **Multithreaded** – can run many tasks at the same time.
- **High performance** – JIT (Just-In-Time) compiler.
- **Simple** – no pointers, no operator overloading, no multiple inheritance of classes.
- **Portable, Distributed, Dynamic** – works across networks and loads classes at runtime.

Q4. What is WORA?

Ans: **WORA = Write Once, Run Anywhere.** You write and compile code once into bytecode (.class file). That same .class file can run on Windows, Linux or Mac, as long as a JVM is installed.

Q5. How does Java achieve platform independence?

Ans: Java source code (.java) is compiled by `javac` into **bytecode** (.class), not into machine code. Bytecode is neutral. Each OS has its own JVM, which converts bytecode into that machine's native instructions.

```
Hello.java --javac--> Hello.class (bytecode) --JVM--> runs on Windows / Linux / Mac
```

Q6. If you compile Java code on Windows, can you run it on Linux?

Ans: Yes. The .class file (bytecode) is platform independent. You only need a JVM (JRE) on Linux. You do not need to recompile.

Q7. Is Java platform independent or is the JVM platform independent?

Ans: **Java (bytecode) is platform independent, but the JVM is platform dependent.** Every OS needs its own JVM build. That is why the same bytecode works everywhere.

Q8. Is Java a compiled or an interpreted language?

Ans: Both. Source code is **compiled** to bytecode by javac, and the bytecode is then **interpreted** (and JIT-compiled) by the JVM at runtime.

Q9. What is bytecode?

Ans: Bytecode is the intermediate, machine-independent code stored in a .class file. The JVM reads and executes it.

Kodewala Java Interview

Q10. What is the JIT compiler?

Ans: Just-In-Time compiler is part of the JVM. It converts frequently used bytecode ("hot spots") into native machine code at runtime, so the program runs faster than pure interpretation.

Q11. Why is Java called "Robust"?

Ans: Because it handles errors well: strong type checking, exception handling, automatic garbage collection (no manual memory freeing), and no pointer errors.

Q12. Why is Java considered secure?

Ans: No explicit pointers, bytecode verifier checks code before running, class loaders separate code, and programs run inside the JVM sandbox.

Q13. Is Java a 100% object-oriented language? Why or why not?

Ans: **No.** Java has 8 primitive types (int, char, etc.) which are not objects, and it has `static` members that work without objects. A pure OOP language treats everything as an object.

Q14. What are the four pillars (principles) of OOP?

- **Encapsulation** – bundle data and methods; hide data using private.
- **Inheritance** – one class acquires properties of another.
- **Polymorphism** – one thing, many forms (overloading, overriding).
- **Abstraction** – show only essential details, hide the implementation.

Q15. What is the difference between a Class and an Object?

Ans: A **class** is a blueprint (template). An **object** is a real instance created from that blueprint. Example: `Car` is a class, your red Honda is an object.

Q16. Why doesn't Java support pointers?

Ans: For safety and simplicity. Pointers cause memory leaks and security issues. Java uses **references** instead, and the JVM manages memory.

Q17. Why doesn't Java support multiple inheritance with classes?

Ans: To avoid the **diamond problem** (confusion when two parent classes have the same method). Java allows multiple inheritance through **interfaces**.

Q18. What is Garbage Collection?

Ans: The JVM automatically finds objects that are no longer used and frees their memory. The programmer does not need to free memory manually.

Q19. What is the difference between Java and C++?

- Java is platform independent (bytecode); C++ is compiled to platform-specific machine code.
- Java has no pointers and no manual memory management; C++ has both.
- Java has no multiple inheritance of classes and no operator overloading; C++ supports them.
- Java has automatic garbage collection; C++ does not.

Q20. What is the difference between Java and JavaScript?

Ans: They are completely different languages. Java is a compiled, general-purpose, class-based language that runs on the JVM. JavaScript is a scripting language mainly used in browsers (and Node.js).

Q21. What are the editions of Java?

- **Java SE** (Standard Edition) – core Java.
- **Java EE / Jakarta EE** (Enterprise Edition) – large web/enterprise apps.

Kodewala Java Interview

- **Java ME** (Micro Edition) – small and embedded devices.

Q22. What is an LTS version of Java?

Ans: LTS = **Long Term Support**. These versions get updates and support for many years. Common LTS versions: Java 8, 11, 17, 21 (and 25). Companies prefer LTS versions in production.

Q23. Where is Java used in real life?

Ans: Web applications (Spring Boot), Android apps, banking and enterprise software, big-data tools, games, and cloud/microservices.

Topic 2: First Java Program (Pages 5–8)

Hello World, main(), System.out.println(), Compile and Run

Q24. Write a simple Hello World program in Java.

Ans: The class name must match the file name (Hello.java).

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Q25. How do you compile and run a Java program?

- Compile: `javac Hello.java` → creates Hello.class
- Run: `java Hello` (no .class extension)

Q26. Explain every word of `public static void main(String[] args)`.

- **public** – JVM can call it from outside the class.
- **static** – JVM can call it without creating an object.
- **void** – it returns nothing to the JVM.
- **main** – the fixed name where the JVM starts execution.
- **String[] args** – array to receive command-line arguments.

Q27. Why is the main method static?

Ans: The JVM starts the program before any object exists. A static method belongs to the class, so the JVM can call it directly using the class name, without creating an object.

Q28. Why is the main method public?

Ans: The JVM is outside your class, so main must be accessible to it. If it is not public, the JVM cannot call it.

Q29. Why is the return type of main void?

Ans: Once the program ends, there is nobody to receive a returned value. The JVM does not expect any value from main.

Q30. Can we run a Java program without a main method?

Ans: No (Java 7 and later). You get an error such as "Main method not found". In very old versions (Java 6 and earlier), a `static {}` block could run before the error appeared.

Q31. Can we change the order of `public static` in main?

Ans: Yes. `static public void main(String[] args)` also works. Modifier order does not matter.

Kodewala Java Interview

Q32. What other forms of the main method signature are valid?

- `public static void main(String... args)` (varargs)
- `public static void main(String args[])` (array brackets after the name)
- `public static void main(final String[] args)`

The argument name can be anything (args, a, input).

Q33. Can we overload the main method?

Ans: Yes, you can create other methods named main with different parameters. But the JVM only starts with `main(String[] args)`.

Q34. What happens if main is not static, or its return type is not void?

Ans: The code compiles, but at runtime the JVM reports an error (main method must be static / must return void) and does not run the program.

Q35. Can the main method throw an exception?

Ans: Yes. You can write `public static void main(String[] args) throws Exception`.

Q36. Can main be final or synchronized?

Ans: Yes, both are allowed. It still works as the entry point.

Q37. What is the purpose of `String[] args`?

Ans: It receives command-line arguments passed while running the program, for example `java Hello John 25`.

Q38. Explain System.out.println().

- **System** – a final class in the `java.lang` package.
- **out** – a static variable of System, of type `PrintStream` (the console output).
- **println()** – a method of PrintStream that prints text and moves to a new line.

Q39. What is the difference between print(), println() and printf()?

- `print()` – prints without a new line.
- `println()` – prints and moves to the next line.
- `printf()` – prints formatted text, e.g. `System.out.printf("Age: %d", 25);`

Q40. Why don't we need to import the System class?

Ans: Because it is in `java.lang`, and the `java.lang` package is imported automatically in every Java program.

Q41. Is the file name and class name always the same?

Ans: Only for a **public class**: the file name must exactly match the public class name. For a non-public class, the file name can be different. A file can have only one public class.

Q42. How many .class files are created for one .java file?

Ans: One .class file is created **for each class** defined in the file (including interfaces and inner classes). If a file has 3 classes, you get 3 .class files.

Q43. Is Java case sensitive?

Ans: Yes. `Main` and `main` are different. `String` must start with capital S and `System` with capital S.

Q44. What is the difference between a compile-time error and a runtime error?

- **Compile-time error** – found by javac before running (syntax mistakes, missing semicolon, type mismatch).

Kodewala Java Interview

- **Runtime error (exception)** – happens while the program runs (divide by zero, null pointer).

Q45. Can we have more than one main method in a project?

Ans: Yes, one in each class. When you run the program you choose which class to start with.

Q46. Where does Java program execution start?

Ans: From the `main()` method of the class you run. Static blocks of that class run before main.

Topic 3: JDK, JRE & JVM (Pages 9–10)

Development and Execution Environment

Q47. What is the difference between JDK, JRE and JVM?

- **JVM** – runs the bytecode.
- **JRE** – JVM + core libraries; needed to *run* Java programs.
- **JDK** – JRE + development tools (javac, debugger, etc.); needed to *write and compile* Java programs.

Simple formula: **JDK = JRE + Dev Tools, JRE = JVM + Libraries.**

Q48. What is JVM?

Ans: Java Virtual Machine is a software engine that loads .class files, verifies them, and runs the bytecode by converting it to machine code for the current OS.

Q49. What is JRE?

Ans: Java Runtime Environment gives everything required to run a Java application: the JVM plus standard class libraries (like java.lang, java.util).

Q50. What is JDK?

Ans: Java Development Kit is the full package for developers. It contains the JRE, the compiler `javac`, the launcher `java`, `jar`, `javadoc`, `jdb` (debugger) and other tools.

Q51. Is the JVM platform dependent or independent?

Ans: **Platform dependent.** Windows, Linux and Mac each have their own JVM. The bytecode it runs is platform independent.

Q52. What are the main parts (architecture) of the JVM?

- **Class Loader** – loads .class files into memory.
- **Runtime Data Areas** – Method Area, Heap, Stack, PC Register, Native Method Stack.
- **Execution Engine** – Interpreter, JIT compiler, Garbage Collector.

Q53. What is the role of the Class Loader?

Ans: It loads classes into the JVM when they are first needed (Bootstrap, Extension/Platform and Application class loaders).

Q54. Which memory areas does the JVM have?

- **Heap** – objects are stored here (shared by all threads).
- **Stack** – method calls and local variables (one per thread).
- **Method Area / Metaspace** – class data and static variables.
- **PC Register** – address of the current instruction.
- **Native Method Stack** – for native (non-Java) code.

Kodewala Java Interview

Q55. What are the order of steps from writing code to running it?

Ans: Write `.java` → compile with `javac` → get `.class` bytecode → Class Loader loads it → Bytecode Verifier checks it → Interpreter/JIT executes it.

Q56. Can we install JRE without JDK?

Ans: Yes, in older versions a standalone JRE could be installed to only run programs. But to compile code, JDK is required. Since Java 11, standalone JRE downloads are no longer provided by Oracle; the JDK is installed (or a custom runtime is built with `jlink`).

Q57. Can we run Java programs with only the JDK and no separate JRE?

Ans: Yes. JDK already contains the JRE inside it.

Q58. An application runs fine on one system but fails on another. Both have JRE installed. What could be the problem?

Ans: Common causes:

- **Different Java version** (e.g., compiled with Java 17, running on Java 8 → `UnsupportedClassVersionError`).
- Different vendor/JVM behaviour or missing libraries/JARs (classpath problem).
- Wrong environment variables (`JAVA_HOME`, `PATH`, `CLASSPATH`).
- OS-specific code (file paths like `C:\` vs `/home/`).
- Different memory settings, permissions or config files.

Q59. What is JAVA_HOME and the PATH variable?

- **JAVA_HOME** – points to the JDK installation folder.
- **PATH** – lets the OS find `javac` and `java` commands from any folder.

Q60. What is the command to check the Java version?

Ans: `java -version` (runtime) and `javac -version` (compiler).

Q61. What is the difference between javac and java?

Ans: `javac` is the **compiler** (turns .java into .class). `java` is the **launcher** (starts the JVM and runs the .class).

Q62. Which JDK is better, Oracle JDK or OpenJDK?

Ans: OpenJDK is the free open-source reference implementation. Oracle JDK is built from it with commercial support. Functionally they are almost the same for learning and most projects.

Topic 4: Command-Line Arguments (Pages 11–12)

main(String[] args), Dynamic Input

Q63. What are command-line arguments?

Ans: Values passed to a Java program at the time of running it. The JVM puts them into the `String[] args` array of the main method.

```
java Demo Rahul 25
// args[0] = "Rahul", args[1] = "25"
```

Q64. How do you read command-line arguments in a program?

Ans: Use the array index: `args[0]`, `args[1]` ... and `args.length` for the count.

Kodewala Java Interview

```
public class Demo {
    public static void main(String[] args) {
        System.out.println("Count: " + args.length);
        for (String a : args) {
            System.out.println(a);
        }
    }
}
```

Q65. What is the data type of command-line arguments?

Ans: Always **String**, even if you pass numbers. Convert them when needed, e.g. `Integer.parseInt(args[0])`.

Q66. How do you add two numbers passed from the command line?

Ans: Convert both Strings to int first.

```
int a = Integer.parseInt(args[0]);
int b = Integer.parseInt(args[1]);
System.out.println("Sum = " + (a + b));
```

Q67. What happens if you read args[0] but pass no arguments?

Ans: You get **ArrayIndexOutOfBoundsException**, because the args array is empty (length 0).

Q68. How do you make sure arguments are passed? How do you warn the user if they forgot?

Ans: Check `args.length` before using the array.

```
if (args.length == 0) {
    System.out.println("Please enter the input");
    return;   // stop the program
}
```

Q69. What happens if you pass text like "abc" and convert it with Integer.parseInt()?

Ans: You get **NumberFormatException**. Handle it with try-catch.

```
try {
    int n = Integer.parseInt(args[0]);
} catch (NumberFormatException e) {
    System.out.println("Please pass a valid number");
}
```

Q70. Can args be null? What is its length if nothing is passed?

Ans: args is never null. If nothing is passed, it is an empty array and `args.length == 0`.

Q71. How are arguments separated? What if an argument contains spaces?

Ans: Arguments are separated by spaces. To pass one value that has spaces, put it in double quotes: `java Demo "Hello World"` → args[0] = Hello World.

Q72. Does args[0] contain the class name like in C/C++?

Ans: No. In Java, args[0] is the first actual argument. The class name is not included.

Q73. Why do we use command-line arguments in real projects?

Ans: To make programs flexible without changing code: file path, environment name (dev/test/prod), port number, or config values can change at runtime.

Q74. You are developing a utility to read a file, and the path may change in future. What is your approach?

Ans: Pass the file path as a command-line argument (or use a config file/environment variable) instead of hard-coding it. Validate that it is provided and that the file exists.

Q75. What is the difference between command-line arguments and Scanner input?

- **Command-line args** – given before the program starts, one time, good for configuration.
- **Scanner** – read while the program is running, interactive, can ask many times.

Q76. How can we pass command-line arguments in an IDE (Eclipse/IntelliJ)?

Ans: Open Run Configuration → Arguments (Program arguments) → type the values separated by spaces.

Q77. Can we change the values of args inside the program?

Ans: Yes, array elements can be changed because args is a normal array (unless declared final reference, but elements are still changeable).

Topic 5: Identifiers & Naming Conventions (Pages 13–17)

Rules, Reserved Words

Q78. What is an identifier in Java?

Ans: The name given to a variable, class, method, package or interface. In `int balance = 10;`, `balance` is an identifier.

Q79. What are the rules for writing identifiers?

- Allowed characters: letters (A–Z, a–z), digits (0–9), dollar `$` and underscore `_`.
- **Cannot start with a digit.**
- **Cannot be a reserved word** (keyword).
- No spaces or special characters like `# @ - %`.
- Case sensitive (`age` and `Age` are different).
- No length limit.

Q80. Which of these are valid identifiers: `myVar`, `2name`, `_count`, `$price`, `my-var`, `class`, `Kodewala#`?

- ✓Valid: `myVar`, `_count`, `$price`
- ✗Invalid: `2name` (starts with digit), `my-var` (hyphen), `class` (keyword), `Kodewala#` (special character)

Q81. What are reserved words (keywords)?

Ans: Words with special meaning to the compiler (like `class`, `public`, `static`, `if`, `for`, `int`). They cannot be used as identifiers. Java has about 53 reserved words (50 keywords + 3 literals: `true`, `false`, `null`).

Q82. What are `goto` and `const` in Java?

Ans: They are reserved words, but they are **not used** in Java. You cannot use them as names either.

Q83. Are `true`, `false` and `null` keywords?

Ans: Technically they are **literals**, but they are reserved, so you cannot use them as identifiers.

Q84. Can we use $ or _ in names? Can a name be just _?

Ans: Yes, `$` and `_` are allowed inside or at the start of names. But a single underscore `_` alone is not allowed as a name from Java 9 onwards.

Q85. Is `main` a keyword? Can we use `String` as a variable name?

Ans: `main` is not a keyword (it is just a special method name). `String` is a class name, not a keyword, so `int String = 5;` compiles — but it is a bad practice.

Kodewala Java Interview

Q86. Can we use var as an identifier?

Ans: `var` (Java 10) is a "reserved type name", not a keyword. A variable can still be named `var`, but a class cannot. Avoid it.

Q87. Can identifiers contain Unicode characters?

Ans: Yes, Java supports Unicode letters, so names like □□□ are valid, but it is not recommended for readability.

Q88. What are the Java naming conventions?

Element	Convention	Example
Class / Interface	PascalCase (start capital)	BankAccount, Runnable
Method	camelCase, usually a verb	getBalance(), calculateTax()
Variable	camelCase, meaningful	studentName, totalMarks
Constant	UPPER_CASE with underscores	MAX_SIZE, PI
Package	all lowercase	com.kodewala.app

Q89. Are naming conventions compulsory?

Ans: No, the compiler does not enforce them. But all teams follow them for clean, readable and maintainable code, and interviewers expect them.

Q90. Why are meaningful names important?

Ans: They make code self-explanatory, easier to maintain and debug. Example: `int d;` is unclear; `int daysSinceJoining;` is clear.

Q91. Count the identifiers in this code: `public class Kodewala { public static void main(String[] args) { int balance = 1200; String location = "Bangalore"; } }`

Ans: 6 identifiers: `Kodewala` (class), `main` (method), `String` (predefined class), `args`, `balance`, `location` (variables).

Q92. What is the difference between a keyword and an identifier?

Ans: A keyword is predefined by Java with a fixed meaning. An identifier is a name chosen by the programmer.

Topic 6: Primitive Data Types (Pages 18–20)

byte, short, int, long, float, double, char, boolean

Q93. What are primitive data types? How many are there?

Ans: Basic built-in types that store the actual value directly (not an address). Java has **8**: `byte, short, int, long, float, double, char, boolean`.

Q94. Give the size, range and default value of each primitive type.

Type	Size	Range / Values	Default
byte	1 byte	-128 to 127	0
short	2 bytes	-32,768 to 32,767	0

Type	Size	Range / Values	Default
int	4 bytes	about -2.1 billion to 2.1 billion	0
long	8 bytes	about -9.2×10^18 to 9.2×10^18	0L
float	4 bytes	~7 decimal digits precision	0.0f
double	8 bytes	~15 decimal digits precision	0.0d
char	2 bytes	0 to 65,535 (Unicode)	'\u0000'
boolean	JVM dependent (1 bit logically)	true / false	false

Q95. What is the default data type of an integer literal and a decimal literal?

Ans: Integer literal like `10` is **int**. Decimal literal like `10.5` is **double**.

Q96. Why do we write L for long and f for float?

Ans: Because plain numbers default to int/double. `long x = 5000000000L;` needs `L` as the number is too big for int. `float f = 3.14f;` needs `f`, otherwise `3.14` is double and gives a compile error.

Q97. What happens if you write `byte b = 130;`?

Ans: Compile-time error, because 130 is outside the byte range (-128 to 127). You would need an explicit cast `(byte) 130`, which gives -126.

Q98. What is the difference between float and double?

Ans: float is 4 bytes with about 7 digits of precision. double is 8 bytes with about 15 digits. double is the default for decimals and is preferred.

Q99. Why is char 2 bytes in Java (and 1 byte in C)?

Ans: Java uses **Unicode (UTF-16)**, so one char can hold characters from any language, not only English.

Q100. Can a char store a number? What is `char c = 65;`?

Ans: Yes. char is stored as a number internally. `char c = 65;` stores 'A'. `c + 1` gives an int (66).

Q101. What is the size of boolean? Can we assign 0 or 1 to it?

Ans: The size is not exactly defined by the Java specification. **A boolean accepts only ¦true¦ or ¦false¦**, not 0 or 1 (unlike C).

Q102. What is type casting? Explain widening and narrowing.

- **Widening (implicit)** – small to big type, automatic: byte → short → int → long → float → double.
- **Narrowing (explicit)** – big to small type, needs a cast and may lose data.

```
int i = 100;
double d = i;        // widening, automatic
double x = 9.78;
int y = (int) x;     // narrowing, y = 9 (decimal lost)
```

Q103. What is integer overflow?

Ans: When a value goes beyond the type's range, it wraps around instead of giving an error.

Kodewala Java Interview

```
int max = Integer.MAX_VALUE;   // 2147483647
System.out.println(max + 1);    // -2147483648
```

Q104. What is the result of `byte a = 10, b = 20; byte c = a + b;`?

Ans: Compile error. byte, short and char are promoted to **int** in arithmetic, so `a + b` is int. Fix: `byte c = (byte)(a + b);`.

Q105. Why is `0.1 + 0.2 == 0.3` false?

Ans: float/double cannot store some decimals exactly (binary representation). The result is 0.30000000000000004. For money use **BigDecimal**, and compare doubles with a small tolerance.

Q106. In a large application where memory is a concern, how do you decide between int and long?

Ans: Use **int** when the value will stay within about 2.1 billion (it uses half the memory). Use **long** only when values can be larger (timestamps, big IDs, large counts). Do not use long everywhere "just in case" in huge arrays/collections.

Q107. You track millions of users with large numerical identifiers. Which data type will you choose for user ID?

Ans: **long** (8 bytes), because millions or billions of users may exceed int's limit. For globally unique/distributed IDs, UUID or String can also be used.

Q108. What are wrapper classes? Name them.

Ans: Object forms of primitives: `Byte, Short, Integer, Long, Float, Double, Character, Boolean`. They are needed for collections (`ArrayList<Integer>`) and give utility methods.

Q109. What are autoboxing and unboxing?

Ans: **Autoboxing** = automatic primitive → wrapper (`Integer a = 5;`). **Unboxing** = wrapper → primitive (`int b = a;`).

Q110. What is the Integer cache?

Ans: Integer objects from **-128 to 127** are cached, so `Integer a = 100, b = 100; a == b` is true, but for 1000 it is false. Always compare wrapper objects with `equals()`.

Q111. Can we use underscores in numbers?

Ans: Yes (Java 7+), for readability: `int salary = 1_00_000;` . You can also write binary `0b101`, octal `017` and hex `0xFF` literals.

Q112. Is String a primitive data type?

Ans: No. String is a class (non-primitive/reference type).

Q113. Which primitive type would you use for money/salary calculation?

Ans: Avoid float/double for exact money. Use `BigDecimal` (or store amount in paise/cents as long).

Topic 7: Non-Primitive Data Types (Pages 21–22)

String, Arrays, Reference Types, null

Q114. What are non-primitive (reference) data types?

Ans: Types that store a **reference (memory address)** to the actual object, not the value itself. Examples: `String`, arrays, classes, interfaces, enums, wrapper classes.

Kodewala_Java_Interview

Q115. What is the difference between primitive and non-primitive types?

Point	Primitive	Non-Primitive
Stores	Actual value	Reference (address) to object
Defined by	Java (8 fixed types)	Java or programmer (classes)
Default value	0, false, etc.	null
Size	Fixed	Not fixed, can grow
Methods	No methods	Can have methods
Can be null	No	Yes
Memory	Stack (as local variable)	Reference on stack, object on heap

Q116. What is the default value of a non-primitive variable?

Ans: **null** (for fields/instance variables and array elements). Local reference variables have no default and must be initialised before use.

Q117. What is null in Java?

Ans: A special literal meaning "no object". A reference variable holding null points to nothing.

Q118. What is NullPointerException? How do you avoid it?

Ans: It occurs when you use a reference that is null (e.g., call a method on it). Avoid it by checking `if (obj != null)`, using `"text".equals(s)`, initialising variables, or using `Optional`.

```
String name = null;
System.out.println(name.length());   // NullPointerException
```

Q119. Can we assign null to a primitive?

Ans: No. `int a = null;` is a compile error. You can assign null to `Integer` (the wrapper).

Q120. How do you check if a non-primitive value is null?

Ans: Use the `==` operator.

```
String s = null;
if (s == null) {
    System.out.println("s is null");
}
```

Q121. What happens when you print a null String?

Ans: It prints the text `null`. Example: `String s = null; System.out.println(s);` → prints null (no error), but `s.length()` throws NPE.

Q122. Is an array a primitive or non-primitive type?

Ans: Non-primitive. Arrays are objects, even an array of ints is an object stored in the heap.

Q123. Where are references and objects stored in memory?

Ans: The reference variable lives on the **stack** (if local); the actual object lives on the **heap**.

```
Student s = new Student();
// s  -> stack (reference)
// new Student() -> heap (object)
```

Kodewala Java Interview

Q124. Is Java pass-by-value or pass-by-reference?

Ans: Always **pass-by-value**. For objects, the value copied is the reference (address). So the method can change the object's content, but cannot make the caller's variable point to a new object.

Q125. What does `null instanceof String` return?

Ans: **false**. instanceof is always false for null.

Q126. How do two reference variables work when you do `Student s2 = s1;`?

Ans: Both point to the **same object**. Changing the object through s2 also changes it for s1. No copy is made.

Q127. Can a non-primitive have a variable size?

Ans: Yes. Types like String, ArrayList and user-defined objects can grow or hold different amounts of data, unlike fixed-size primitives.

Topic 8: Java Variables (Pages 23–27)

Local, Instance, Static Variables

Q128. What is a variable?

Ans: A named memory location used to store data. Syntax: `dataType name = value;` e.g. `int age = 25;`

Q129. What are the types of variables in Java?

- **Local variable** – declared inside a method/block/constructor.
- **Instance variable** – declared inside a class, outside methods, without `static`.
- **Static variable** – declared inside a class with the `static` keyword.

Q130. Explain local, instance and static variables with an example.

```
public class Demo {
    int id = 1;                 // instance variable (each object has its own)
    static String company = "ABC"; // static variable (one copy, shared)

    void show() {
        int x = 10;             // local variable (only inside this method)
        System.out.println(x + id + company);
    }
}
```

Q131. What is the difference between instance variable and static variable?

Point	Instance variable	Static variable
Keyword	No static	Declared with static
Copies	One copy per object	One copy per class (shared)
Created when	Object is created (new)	Class is loaded
Memory	Heap (inside object)	Method Area / Metaspace
Access	Using object	Using class name (or object)
Destroyed when	Object is garbage collected	Class is unloaded

Q132. What are the default values of local, instance and static variables?

- **Instance and static** variables get default values (0, false, null).
- **Local variables get NO default value**. You must initialise them before use, otherwise you get a compile error.

Q133. Why don't local variables get default values?

Ans: They live on the stack for a short time. Java forces the programmer to initialise them to avoid using garbage values by mistake.

Q134. How does the JVM manage memory for instance and static variables?

Ans: Instance variables are stored in the **heap** inside each object. Static variables are stored in the **Method Area (Metaspace in Java 8+)** once per class, and shared by all objects. Local variables are in the **stack frame** of the method.

Q135. Can a static method access an instance variable directly? Why or why not?

Ans: **No.** A static method belongs to the class and can run without any object, so there is no specific object whose instance variable to use. It can access them only through an object reference.

```
class A {
    int x = 5;
    static void test() {
        // System.out.println(x);   // compile error
        A obj = new A();
        System.out.println(obj.x);  // OK
    }
}
```

Q136. Can an instance method access a static variable?

Ans: Yes. Static variables belong to the class, and instance methods can always access them.

Q137. How would you use a static variable to count the total number of objects created?

Ans: Increase the static counter inside the constructor.

```
class Student {
    static int count = 0;
    Student() {
        count++;
    }
}
// new Student(); new Student();  -> Student.count = 2
```

Q138. Design a bank account class: each account has its own balance, all accounts share one interest rate.

Ans: balance is an **instance** variable; interestRate is a **static** variable.

```
class BankAccount {
    double balance;                    // instance - different for each account
    static double interestRate = 4.5;  // static - same for all accounts

    BankAccount(double balance) {
        this.balance = balance;
    }
    double interest() {
        return balance * interestRate / 100;
    }
}
```

Q139. What happens if an instance variable and a static variable have the same name in a class?

Ans: Compile-time error: **variable already defined**. A class cannot have two fields with the same name, even if one is static.

Q140. What happens if a local variable has the same name as an instance variable?

Ans: It is allowed. The local variable **shadows** (hides) the instance variable inside that method. Use `this.name` to access the instance variable.

```
class A {
    int x = 10;
    void show() {
        int x = 20;
        System.out.println(x);       // 20 (local)
        System.out.println(this.x);  // 10 (instance)
    }
}
```

Q141. What is the `this` keyword?

Ans: A reference to the **current object**. It is used to differentiate instance variables from parameters/local variables with the same name, and to call other constructors/methods of the same class.

Q142. How do you access a static variable?

Ans: Using the class name: `ClassName.variable`, e.g. `BankAccount.interestRate`. It can also be accessed through an object, but this is not recommended.

Q143. What is a final variable? How is it different from a normal variable?

Ans: Once assigned, a `final` variable cannot be changed. Constants are written as `static final`, e.g. `static final double PI = 3.14;`. For a final reference, the reference cannot change, but the object's content can.

Q144. What is the `var` keyword?

Ans: Introduced in Java 10. It lets the compiler infer the type of a **local** variable: `var name = "Java";`. The type is still fixed at compile time (not dynamic typing). It cannot be used for fields or method parameters.

Q145. Can we declare a local variable as static or public?

Ans: No. Local variables cannot have access modifiers or `static`. Only `final` is allowed.

Q146. Can we declare a variable without initialising it?

Ans: Yes for instance/static variables (they get defaults). For local variables you can declare it, but you must assign a value before you use it.

Q147. What is a static block?

Ans: A block of code `static { ... }` that runs once when the class is loaded, before main. Used to initialise static variables.

Q148. Are static variables thread-safe?

Ans: No. Since they are shared by all objects/threads, simultaneous changes can cause wrong results. Use `synchronized`, atomic classes or avoid shared mutable state.

Kodewala Java Interview

Topic 9: Scope of Variables (Pages 28–30)

Member Variables, Local Scope, Lifecycle

Q149. What is the scope of a variable?

Ans: The part of the program where a variable can be accessed (seen and used).

Q150. What are member variables? How do they differ from local variables?

Ans: **Member variables** are declared inside a class but outside any method (instance and static variables). They are accessible in the whole class and live as long as the object/class. **Local variables** are declared inside a method/block, accessible only there, and die when it ends.

Q151. Explain the scope and lifecycle of local variables.

- **Scope**: only inside the method/block/loop where declared (from declaration to the closing brace).
- **Lifecycle**: created when the method (block) starts executing, stored on the stack, destroyed when it finishes.

Q152. Explain the lifecycle of instance and static variables.

- **Instance**: created when the object is created (`new`), removed when the object is garbage collected.
- **Static**: created when the class is loaded, removed when the class is unloaded (usually when the program ends).

Q153. Why can't local variables have access modifiers in Java?

Ans: Access modifiers decide visibility across classes and packages. A local variable is visible only inside its own method/block, so access modifiers make no sense for it.

Q154. Can we access a variable declared inside a block outside of it?

Ans: No. It gives a compile error "cannot find symbol".

```
if (true) {
    int a = 10;
}
// System.out.println(a);  // error: a is out of scope
```

Q155. What is the scope of the loop variable in `for (int i = 0; ...)`?

Ans: Only inside the for loop. After the loop, `i` is not available. To use it later, declare it before the loop.

Q156. Can we declare two local variables with the same name in the same method?

Ans: Not in the same scope or in a nested inner block. Java gives a compile error. But two separate blocks (e.g., two different for loops) can each use `i`.

Q157. What is variable shadowing?

Ans: When a local variable (or parameter) has the same name as a member variable, the local one hides the member one inside that method. Use `this.variable` to reach the member.

Q158. Can a method parameter be used outside the method?

Ans: No. Parameters are local to the method, with scope only inside it.

Q159. What happens to local variables when a method finishes?

Ans: The stack frame is removed and the local variables are destroyed. Their values are lost (objects they referenced may be garbage collected).

Kodewala Java Interview

Q160. What is block scope?

Ans: A variable declared inside curly braces `{ }` (if, for, while, plain block, try) is accessible only inside those braces.

Q161. Can a local variable be used in a lambda or inner class?

Ans: Only if it is **final or effectively final** (its value is never changed after assignment).

Q162. What error do you get for using an uninitialised local variable?

Ans: "variable might not have been initialized" (compile-time error).

Q163. Can a static variable be accessed from anywhere in the class?

Ans: Yes. Inside any method (static or instance) in that class. From outside, depending on its access modifier (public, etc.) using `ClassName.var`.

Topic 10: Java Operators (Pages 31–33)

Assignment, Relational, Logical Operators

Q164. What are the types of operators in Java?

- **Arithmetic**: `+ - * / %`
- **Assignment**: `= += -= *= /= %=`
- **Relational (comparison)**: `== != > < >= <=`
- **Logical**: `&& || !`
- **Unary**: `++ -- + - !`
- **Bitwise**: `& | ^ ~ << >> >>>`
- **Ternary**: `condition ? a : b`
- **instanceof**: type check

Q165. What is the assignment operator? What are compound assignment operators?

Ans: `=` stores a value in a variable. Compound ones combine an operation with assignment: `a += 5` means `a = a + 5`.

Q166. What are relational operators?

Ans: They compare two values and return a boolean (true/false): `==, !=, >, <, >=, <=`. Example: `5 > 3` → true.

Q167. What are logical operators?

Ans: They combine boolean conditions:

- **&& (AND)** – true if both are true.
- **|| (OR)** – true if at least one is true.
- **! (NOT)** – reverses true/false.

Q168. What happens if the first condition in a logical AND (&&) is false?

Ans: Java uses **short-circuit evaluation**: the second condition is **not evaluated**, since the result must be false. Similarly in `||` if the first is true, the second is skipped.

```
int a = 5;
if (a > 10 && ++a > 0) { }
System.out.println(a);   // 5, because ++a never ran
```

Q169. What is the difference between && and & (or || and |)?

Ans: `&&` / `||` are short-circuit (may skip the second condition). `&` / `|` always evaluate **both** sides, and also work as bitwise operators on numbers.

Kodewala Java Interview

Q170. Difference between = and ==?

Ans: = assigns a value. == compares two values. `if (a = 5)` is a compile error for int (it needs boolean).

Q171. Difference between == and equals()?

Ans: For primitives, == compares values. For objects, == compares **references** (same object?), while `equals()` compares **content** (if overridden, like in String).

Q172. Explain pre-increment and post-increment.

- **Post** a++: use the value first, then increase.
- **Pre** ++a: increase first, then use the value.

```
int a = 5;
int b = a++;   // b = 5, a = 6
int c = ++a;   // a = 7, c = 7
```

Q173. What is the output of int i = 5; i = i++; ?

Ans: i stays **5**. The old value (5) is remembered, i becomes 6, then the old value 5 is assigned back to i.

Q174. What does the ternary operator do?

Ans: It is a short form of if-else: `condition ? valueIfTrue : valueIfFalse`.

```
int max = (a > b) ? a : b;
```

Q175. What is the difference between / and %?

Ans: / gives the quotient, % gives the remainder. `10 / 3 = 3` and `10 % 3 = 1`. With ints, / drops the decimal part.

Q176. What happens when we divide by zero in Java?

- Integers: throws **ArithmeticException** (/ by zero).
- Doubles: gives `Infinity` (or `NaN` for 0.0/0.0) without an exception.

Q177. What is the output of System.out.println(1 + 2 + "3"); and System.out.println("1" + 2 + 3); ?

Ans: First: **33** (1+2 = 3, then "3" joined). Second: **123** (all joined as strings from left to right).

Q178. Is operator overloading supported in Java?

Ans: No, except that + is built-in for both addition and String concatenation.

Q179. What is the operator precedence in Java (basic idea)?

Ans: Order (high to low): unary (`++ -- !`) → `* / %` → `+ -` → relational → equality → && → || → ternary → assignment. Use parentheses `( )` to be clear.

Q180. Write a program to check if a number is divisible by both 3 and 5.

Ans: Use % (relational) with && (logical).

```
int n = 15;
if (n % 3 == 0 && n % 5 == 0) {
    System.out.println("Divisible by 3 and 5");
} else {
    System.out.println("Not divisible");
}
```

Kodewala Java Interview

Q181. Given temperature and humidity (from command line), find if the day is above 30°C and humidity is below 50%.

Ans: Parse both values from args and use &&.

```
double temp = Double.parseDouble(args[0]);
double humidity = Double.parseDouble(args[1]);
if (temp > 30 && humidity < 50) {
    System.out.println("Hot and dry day");
} else {
    System.out.println("Condition not met");
}
```

Q182. How can you swap two numbers without a third variable?

```
int a = 5, b = 10;
a = a + b;   // 15
b = a - b;   // 5
a = a - b;   // 10
```

Q183. What is the output of `byte b = 10; b += 5;`? Does it compile?

Ans: Yes, it compiles. Compound assignment operators do an **implicit cast** back to the variable type. But `b = b + 5;` would give a compile error.

Q184. What are bitwise and shift operators?

Ans: They work on bits of integers: & AND, | OR, ^ XOR, ~ NOT, << left shift (multiply by 2), >> right shift (divide by 2).

Q185. What is the instanceof operator?

Ans: Checks if an object is an instance of a class/interface. Returns true/false. `obj instanceof String` is false if obj is null.

Q186. What is the difference between `a == b` for floats and why is it risky?

Ans: Floating-point values are approximate, so == may give false for values that look equal. Compare using `Math.abs(a - b) < 0.00001`.

Topic 11: User Input — Scanner (Pages 34–36)

nextInt(), nextLine(), Input Validation

Q187. What is the Scanner class?

Ans: A class in `java.util` used to read input from the keyboard (System.in), files or strings.

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Enter age: ");
int age = sc.nextInt();
sc.close();
```

Q188. Which Scanner methods do you know?

Ans: `nextInt()`, `nextLong()`, `nextDouble()`, `nextBoolean()`, `next()` (one word), `nextLine()` (full line), and `hasNext()`, `hasNextInt()` to check input.

Q189. What is the difference between `next()` and `nextLine()`?

Ans: `next()` reads one word (stops at a space). `nextLine()` reads the entire line including spaces, until Enter.

Q190. Why does nextLine() get skipped after nextInt()? How do you fix it?

Ans: `nextInt()` reads only the number and leaves the Enter (newline) in the buffer. The next `nextLine()` reads that leftover newline as an empty line. **Fix**: add an extra `sc.nextLine()` to consume it, or read everything with nextLine() and parse.

```
int age = sc.nextInt();
sc.nextLine();                // consume leftover newline
String name = sc.nextLine();
```

Q191. Is there a nextChar() method?

Ans: No. To read a char use `sc.next().charAt(0)`.

Q192. What is System.in?

Ans: The standard input stream (the keyboard). Scanner wraps it to make reading easy.

Q193. What happens if the user enters text when nextInt() is expected?

Ans: It throws **InputMismatchException**.

Q194. How do you validate user input (e.g., age must be an integer)?

Ans: Use `hasNextInt()` before reading, and ask again if invalid.

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter age: ");
while (!sc.hasNextInt()) {
    System.out.println("Invalid! Please enter a number.");
    sc.next();   // discard the wrong input
}
int age = sc.nextInt();
System.out.println("Age = " + age);
```

Q195. How can you also validate the range (e.g., age 1 to 120)?

```
int age;
do {
    System.out.print("Enter age (1-120): ");
    while (!sc.hasNextInt()) { sc.next(); }
    age = sc.nextInt();
} while (age < 1 || age > 120);
```

Q196. Can you use try-catch to handle wrong input?

Ans: Yes. Catch `InputMismatchException`, clear the bad input with `sc.next()` and ask again.

Q197. Why should we close the Scanner? Any problem with closing it?

Ans: Closing releases the resource. But closing a Scanner on `System.in` also closes System.in, so you cannot read from the keyboard again. Close it once, at the end of the program (or use try-with-resources).

Q198. What is NoSuchElementException in Scanner?

Ans: Thrown when you try to read but there is no more input (e.g., `nextLine()` when input has ended).

Q199. What is the difference between Scanner and BufferedReader?

- **Scanner**: easy, parses ints/doubles directly, slower, not synchronized.
- **BufferedReader**: faster for large input, reads only Strings (you must parse), needs IOException handling.

Q200. What is the difference between Scanner and command-line arguments?

Ans: Command-line args are given before running (one time). Scanner reads while the program is running (interactive).

Q201. How do you read multiple numbers separated by spaces?

Ans: Call `nextInt()` repeatedly: `int a = sc.nextInt(); int b = sc.nextInt();`. Scanner splits input by whitespace by default.

Q202. How do you read a full name with spaces?

Ans: Use `nextLine()`: `String name = sc.nextLine();`.

Q203. Can a Scanner read from a String or a file?

Ans: Yes. `new Scanner("10 20 30")` reads from a String; `new Scanner(new File("a.txt"))` reads from a file.

Q204. Write a program to read two numbers and print the larger one.

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter two numbers: ");
int a = sc.nextInt();
int b = sc.nextInt();
System.out.println("Larger = " + Math.max(a, b));
```

Kodewala Java Interview

PART B — Methods, Control Flow, Arrays & Strings

Pages 37–103

Topic 12: Java Methods (Pages 37–43)

Syntax, Parameters, Return Type, Signature

Q205. What is a method in Java?

Ans: A block of code that performs a specific task and can be reused by calling it. All methods must be inside a class.

Q206. What is the syntax of a method?

```
accessModifier returnType methodName(parameters) {
    // method body (business logic)
}

public int add(int a, int b) {
    return a + b;
}
```

Q207. Why do we need methods?

- **Reusability** – write once, use many times.
- **Readability** – big programs are split into small parts.
- **Maintainability** – change logic in one place.
- **Modularity** – easy testing and teamwork.

Q208. What is a method signature?

Ans: The method **name + the list of parameter types (in order)**. Return type and access modifier are **not** part of the signature. Example: `add(int, int)`. The compiler uses it to identify the method.

Q209. What is the significance of the method signature?

Ans: It identifies a method uniquely. Two methods in the same class cannot have the same signature. It is the base for overloading and overriding.

Q210. What is the difference between parameters and arguments?

Ans: **Parameters** are the variables in the method definition. **Arguments** are the actual values passed when calling it.

```
static int add(int a, int b)   // a, b = parameters
add(5, 10);                      // 5, 10 = arguments
```

Q211. What is the return type? What if a method returns nothing?

Ans: The type of value the method gives back. Use `void` if it returns nothing. A non-void method must return a value of that type using `return`.

Q212. Can a method return more than one value?

Ans: Not directly. Return an array, a List, or an object (class) holding several values.

Q213. What is method overloading?

Ans: Same method name, **different parameters** (number, type or order) in the same class. It is **compile-time polymorphism**. Changing only the return type is not overloading.

```
int add(int a, int b)
double add(double a, double b)
int add(int a, int b, int c)
```

Q214. Can we overload a method by changing only the return type?

Ans: No. It gives a compile error, because the signature (name + parameters) is the same and the compiler cannot decide which one to call.

Q215. What is method overriding?

Ans: A child class gives its own implementation of a method already defined in the parent class (same name, same parameters). It is **runtime polymorphism**.

Q216. Difference between overloading and overriding?

Point	Overloading	Overriding
Where	Same class	Parent and child class
Parameters	Must be different	Must be same
Binding	Compile time	Runtime
Return type	Can differ	Same (or covariant)
static methods	Can be overloaded	Cannot be overridden

Q217. Is Java pass-by-value or pass-by-reference?

Ans: Always **pass-by-value**. Primitives: a copy of the value is passed, so the original does not change. Objects: a copy of the reference is passed, so the object's content can change, but the caller's reference cannot be re-pointed.

```
static void change(int x) { x = 100; }
int a = 10;
change(a);
System.out.println(a);   // still 10
```

Q218. What are varargs?

Ans: Variable-length arguments (`int... nums`) let a method accept any number of arguments of the same type. Internally it is an array. It must be the last parameter, and only one is allowed per method.

```
static int sum(int... nums) {
    int total = 0;
    for (int n : nums) total += n;
    return total;
}
sum(1, 2);  sum(1, 2, 3, 4);
```

Q219. Can we define a method inside another method?

Ans: No. Java does not allow nested methods. (You can use lambdas or inner classes.)

Q220. What is recursion?

Ans: A method calling itself. It must have a **base condition** to stop; otherwise `StackOverflowError` occurs.

```
static int factorial(int n) {
    if (n <= 1) return 1;        // base condition
    return n * factorial(n - 1);
}
```

Q221. Can a method be called without creating an object?

Ans: Yes, if it is `static` (call with the class name). Instance methods need an object.

Q222. What happens if a method has a non-void return type but there is no return statement?

Ans: Compile error: "missing return statement".

Q223. Can a void method use the return keyword?

Ans: Yes, only as `return;` (without a value) to exit the method early.

Q224. Write a method to calculate the total price of items in a shopping cart.

```
static double totalPrice(double[] prices) {
    double total = 0;
    for (double p : prices) {
        total += p;
    }
    return total;
}
```

Q225. Create a method `sendEmail(String recipient, String message)` to simulate sending an email.

```
static void sendEmail(String recipient, String message) {
    System.out.println("Sending email to: " + recipient);
    System.out.println("Message: " + message);
}
```

Q226. What is the difference between a method and a constructor?

Ans: A constructor has the same name as the class, **no return type**, and runs automatically at object creation to initialise it. A method has any name and a return type, and is called explicitly.

Q227. Can we call a method with a different type argument, e.g. pass `int` where `double` is expected?

Ans: Yes, int is automatically widened to double. The reverse (double to int) needs an explicit cast.

Topic 13: Access Modifiers (Pages 44–48)

public, private, protected, default

Q228. What are access modifiers?

Ans: Keywords that control **who can access** a class, variable, method or constructor. Java has four levels: `public, protected, default (no keyword), private`.

Q229. Explain the four access modifiers.

Modifier	Same class	Same package	Subclass (other package)	Other package
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
default	Yes	Yes	No	No
private	Yes	No	No	No

Q230. What is the difference between protected and default?

Ans: **Default** (no keyword): accessible only within the same package. **Protected**: accessible in the same package **and** in subclasses in other packages (through inheritance).

Q231. How do you allow access within the same package and to subclasses in other packages?

Ans: Use the `protected` modifier.

Q232. What is the default access modifier called? Is there a keyword for it?

Ans: It is called **default** or **package-private**. There is no keyword; you simply write nothing. (The `default` keyword in switch and interfaces is different.)

Q233. Which access modifiers can be applied to a top-level class?

Ans: Only **public** or **default** (package-private). A top-level class cannot be private or protected. (Inner/nested classes can use all four.)

Q234. Can a private method be overridden?

Ans: **No.** A private method is not visible to the child class, so it cannot be overridden. If a child defines the same method, it is a new, separate method.

Q235. Can we have a private constructor? Why?

Ans: Yes. Used in **Singleton** pattern (only one object allowed) and utility classes (to stop object creation).

Q236. Why do we make variables private and give getters/setters?

Ans: That is **encapsulation**. It protects data from wrong direct changes, and lets us add validation inside setters.

Q237. Can private members be accessed using an object of the same class?

Ans: Yes. Private means private to the class, so any code inside the same class can access private members of any object of that class.

Q238. Can an overriding method have a weaker access than the parent method?

Ans: No. It can keep the same or increase the visibility (e.g., protected → public) but cannot reduce it.

Q239. Which is the most restrictive and which is the least restrictive modifier?

Ans: Most restrictive: **private**. Least restrictive: **public**. Order: private < default < protected < public.

Q240. Can a protected member be accessed from a different package without inheritance?

Ans: No. In another package it is accessible only through inheritance (in the subclass), not through a plain object of the parent class.

Q241. Why is main declared public?

Ans: So the JVM (outside the class/package) can call it.

Q242. Can we use access modifiers for local variables?

Ans: No. Local variables are limited to their method/block.

Q243. What happens if you try to access a private variable from another class?

Ans: Compile-time error: "has private access in ClassName".

Kodewala Java Interview

Topic 14: Static vs Instance Methods (Pages 49–54)

Method Calling, Object Creation, Static Binding

Q244. What is a static method?

Ans: A method declared with `static`. It belongs to the **class**, not to an object. You call it with the class name, without creating an object: `ClassName.method()`.

Q245. What is an instance method?

Ans: A method without `static`. It belongs to an **object**, so you must create an object to call it. It can use that object's data.

Q246. What is the difference between a static method and an instance method?

Point	Static method	Instance method
Belongs to	Class	Object
Called using	ClassName.method()	object.method()
Object needed	No	Yes
Access	Only static members directly	Static and instance members
this / super	Cannot use	Can use
Overriding	Cannot be overridden (hidden)	Can be overridden
Binding	Static (compile time)	Dynamic (runtime)

Q247. Show how to call static and instance methods.

```
class Demo {
    static void hello() { System.out.println("Static"); }
    void hi()           { System.out.println("Instance"); }

    public static void main(String[] args) {
        Demo.hello();        // static: via class name
        Demo d = new Demo(); // object needed
        d.hi();              // instance: via object
    }
}
```

Q248. Can an instance method be called without creating an object?

Ans: No. You need an object (or you must call it from another instance method of the same class). Calling it from static main without an object gives a compile error.

Q249. Can a static method call an instance method directly?

Ans: No, it needs an object: `new Demo().hi();`. A static method can directly call only other static methods.

Q250. Can an instance method call a static method?

Ans: Yes, directly. Static members are always available.

Q251. Can we override a static method? Why or why not?

Ans: No. Static methods belong to the class and use **static (compile-time) binding**. If the child class declares the same static method, it is **method hiding**, not overriding.

Q252. What is method hiding?

Ans: When a subclass defines a static method with the same signature as the parent's static method. Which one runs depends on the **reference type**, not on the object type.

Q253. What is static binding vs dynamic binding?

- **Static (early) binding** – the method is decided at **compile time** (static, private, final methods, overloading).
- **Dynamic (late) binding** – the method is decided at **runtime** based on the actual object (overriding).

Q254. Can static methods use `this` or `super`?

Ans: No. They are not linked to any object, so there is no current object (this).

Q255. Can we make a static method abstract?

Ans: No. Abstract methods must be overridden, but static methods cannot be overridden.

Q256. Can we call a static method using an object reference or a null reference?

Ans: Yes, it works (even with null) because the compiler replaces it with the class name. But it is bad practice and shows a warning.

Q257. When should you use static methods and when instance methods?

Ans: Use **static** for utility/helper logic that does not depend on object data (`Math.max()`, tax calculation, logging, validations). Use **instance** when the method works with the object's own data (`account.deposit()`, order processing).

Q258. How would you manage global settings in a class using static?

Ans: Keep settings in static variables and provide static methods to get/set them, so all parts of the application share the same values.

```
class AppConfig {
    private static String env = "DEV";
    static String getEnv() { return env; }
    static void setEnv(String e) { env = e; }
}
```

Q259. How are instance methods used to manage object-specific data?

Ans: They work on the instance variables of one object, e.g. `deposit()` updates that account's balance only.

```
class BankAccount {
    double balance;
    void deposit(double amt) { balance += amt; }
}
```

Q260. Where are static methods stored in memory?

Ans: In the **Method Area (Metaspace)** with the class data, not inside any object. Only one copy exists.

Q261. Why is main static?

Ans: So the JVM can call it before any object exists.

Q262. Can a static method be overloaded?

Ans: Yes. Overloading works for static methods (it is resolved at compile time).

Kodewala Java Interview

Q263. Can a static method be final or private?

Ans: Yes, both are allowed. `final` prevents hiding in the child class.

Q264. What is the output? `class A { static void show(){print("A");} } class B extends A { static void show(){print("B");} } A obj = new B(); obj.show();`

Ans: **A**. Static methods are bound to the reference type (A), not the object (B). This proves static methods are not overridden.

Topic 15: If-Else Statements (Pages 55–58)

Conditions, Decision-Making, if-else-if

Q265. What are conditional (decision-making) statements?

Ans: Statements that run different code based on a condition: `if`, `if-else`, `if-else-if`, nested `if`, `switch`.

Q266. Write the syntax of if, if-else and if-else-if.

```
if (marks >= 90) {
    System.out.println("A grade");
} else if (marks >= 60) {
    System.out.println("B grade");
} else {
    System.out.println("Fail");
}
```

Q267. What type must the condition in an if statement be?

Ans: Only **boolean** (true/false). Unlike C, you cannot use 0 or 1. `if (1)` is a compile error.

Q268. What happens if you write `if (a = 5)` instead of `if (a == 5)`?

Ans: For int, a compile error ("int cannot be converted to boolean"). But with boolean, `if (flag = true)` compiles and always assigns true (a common bug).

Q269. Are curly braces required for a single statement in if?

Ans: Not required, but strongly recommended. Without braces only the **first statement** belongs to the if.

Q270. What is a nested if?

Ans: An if statement placed inside another if. The inner one runs only when the outer condition is true.

Q271. What is the dangling else problem?

Ans: When braces are missing, an `else` belongs to the **nearest** `if` — not necessarily the one you intended. Always use braces.

Q272. What is the difference between if-else-if and multiple separate if statements?

Ans: In **if-else-if**, once a condition is true, the rest are skipped. With **separate ifs**, every condition is checked independently.

Q273. What is the difference between if-else and switch?

Ans: if-else can check ranges and complex conditions (`marks > 50`). switch compares one value against fixed constants. switch is cleaner for many fixed choices.

Kodewala Java Interview

Q274. Can we use == to compare Strings in an if condition?

Ans: Avoid it. `==` compares references. Use `str.equals("text")` (or `equalsIgnoreCase`) to compare content.

Q275. Write a program to find the largest of three numbers.

```
int a = 10, b = 25, c = 15;
if (a >= b && a >= c) {
    System.out.println(a + " is largest");
} else if (b >= c) {
    System.out.println(b + " is largest");
} else {
    System.out.println(c + " is largest");
}
```

Q276. Write a program to check if a number is even or odd.

```
int n = 7;
if (n % 2 == 0) System.out.println("Even");
else            System.out.println("Odd");
```

Q277. Write a program to check for a leap year.

Ans: A year is leap if divisible by 400, or divisible by 4 but not by 100.

```
int y = 2024;
if ((y % 400 == 0) || (y % 4 == 0 && y % 100 != 0)) {
    System.out.println("Leap year");
} else {
    System.out.println("Not a leap year");
}
```

Q278. How is the if-else used in real projects?

Ans: Access control (isAdmin, isManager), input validation, business rules (discount, eligibility, tax), and error handling.

Q279. Can the else part be omitted?

Ans: Yes. `else` is optional; a plain `if` alone is valid.

Q280. Can we write an if statement with an empty body?

Ans: Yes (`if (x > 5);`), but a stray semicolon is a common bug: the following block will always run. Avoid it.

Q281. How do you compare double values in an if condition?

Ans: Avoid direct `==`. Use a tolerance: `Math.abs(a - b) < 0.0001`.

Topic 16: Loops (Pages 59–65)

for, while, do-while, nested loops

Q282. What are loops? Name the loops in Java.

Ans: Loops repeat a block of code while a condition is true. Java has: `for`, `while`, `do-while` and the enhanced `for-each` loop.

Kodewala Java Interview

Q283. Write the syntax of for, while and do-while loops.

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}

int j = 1;
while (j <= 5) {
    System.out.println(j);
    j++;
}

int k = 1;
do {
    System.out.println(k);
    k++;
} while (k <= 5);
```

Q284. What is the difference between for loop and while loop? When to use each?

- **for** – use when the number of iterations is **known** (count-based); initialisation, condition and update are in one line.
- **while** – use when it depends on a **condition** and the count is unknown (e.g., reading input until the user types "exit").

Q285. What is the difference between while and do-while?

Ans: **while** checks the condition **first**, so it may run 0 times. **do-while** runs the body **at least once**, and checks the condition afterwards.

Q286. What is the for-each (enhanced for) loop?

Ans: A simple loop to go through every element of an array or collection without an index.

```
int[] nums = {10, 20, 30};
for (int n : nums) {
    System.out.println(n);
}
```

Q287. What are the limitations of the for-each loop?

Ans: You have no index, cannot go backwards, cannot skip elements by steps, and cannot modify the array elements through the loop variable (it is a copy for primitives). Modifying a collection while iterating gives `ConcurrentModificationException`.

Q288. What is an infinite loop? How can you create one?

Ans: A loop that never ends. Examples: `while (true) { }` and `for (;;) { }`. It ends only with `break`, `return` or an exception. A forgotten update (like missing `i++`) also causes it accidentally.

Q289. What is a nested loop?

Ans: A loop inside another loop. The inner loop runs fully for each iteration of the outer loop. Used for patterns, matrices and 2D arrays.

Q290. Print a right-angle star pattern of 5 rows.

```
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

Q291. Write a program to find the sum of all even numbers in an array.

```
int[] arr = {1, 2, 3, 4, 5, 6};
int sum = 0;
for (int n : arr) {
    if (n % 2 == 0) sum += n;
}
System.out.println("Sum = " + sum);   // 12
```

Q292. Write a program to find the factorial of a number.

```
int n = 5;
long fact = 1;
for (int i = 1; i <= n; i++) {
    fact *= i;
}
System.out.println(fact);   // 120
```

Q293. Write a program to print the Fibonacci series.

```
int a = 0, b = 1;
for (int i = 1; i <= 10; i++) {
    System.out.print(a + " ");
    int next = a + b;
    a = b;
    b = next;
}
```

Q294. Write a program to check if a number is prime.

Ans: Check divisibility up to the square root.

```
int n = 29;
boolean prime = n > 1;
for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) { prime = false; break; }
}
System.out.println(prime ? "Prime" : "Not prime");
```

Q295. Write a program to reverse a number and check palindrome.

```
int n = 121, rev = 0, temp = n;
while (temp > 0) {
    rev = rev * 10 + temp % 10;
    temp /= 10;
}
System.out.println(rev == n ? "Palindrome" : "Not palindrome");
```

Q296. Write a program to count the digits (or sum of digits) of a number.

```
int n = 1234, sum = 0;
while (n > 0) {
    sum += n % 10;
    n /= 10;
}
System.out.println(sum);   // 10
```

Q297. Can the loop variable be declared outside the for loop? Can any part of for be empty?

Ans: Yes to both. `int i = 0; for (; i < 5; i++)` is valid. Initialisation, condition and update are all optional.

Q298. Can a for loop have multiple variables?

Ans: Yes. `for (int i = 0, j = 10; i < j; i++, j--)` declares and updates two variables (same type).

Q299. What is a labeled loop?

Ans: A name given to a loop, used with `break label;` or `continue label;` to control an outer loop from inside an inner loop.

Q300. What is the output of `for (int i = 0; i < 3; i++); { System.out.println("Hi"); }`?

Ans: Prints "Hi" **once**. The semicolon after the for makes the loop body empty; the block runs after the loop.

Q301. Which loop is best for traversing an array?

Ans: for-each when you only read values; classic for when you need the index (to modify, reverse, or skip elements).

Topic 17: Switch Statements (Pages 66–68)

Multiple-choice Conditions

Q302. What is a switch statement?

Ans: A statement that selects one block to run from many choices, by matching a value against `case` labels. It is a cleaner alternative to long if-else-if chains.

```
int day = 3;
switch (day) {
    case 1: System.out.println("Mon"); break;
    case 2: System.out.println("Tue"); break;
    case 3: System.out.println("Wed"); break;
    default: System.out.println("Other");
}
```

Q303. Which data types are allowed in a switch?

Ans: `byte, short, int, char`, their wrappers (`Byte, Short, Integer, Character`), `String` (Java 7+) and `enum`. **Not allowed:** `long, float, double, boolean`.

Q304. What happens if you omit the break statement in a switch? (Fall-through)

Ans: Execution continues into the **next case**, whether or not it matches, until a `break` or the end of the switch. This is called **fall-through**.

```
int x = 1;
switch (x) {
    case 1: System.out.println("One");
    case 2: System.out.println("Two");
    default: System.out.println("Default");
}
// Output: One  Two  Default
```

Q305. Is the default case compulsory? Where can it be placed?

Ans: Not compulsory. It runs when no case matches. It can be placed anywhere, but is usually last (if not last, it needs a break).

Q306. Can we have duplicate case labels?

Ans: No. It gives a compile error "duplicate case label".

Q307. Can case values be variables?

Ans: Only **constants** (literals or `final` variables initialised with constant values). A normal variable is not allowed.

Q308. What happens if the switch expression is null?

Ans: A `NullPointerException` is thrown (for String / wrapper / enum).

Q309. Is switch on Strings case sensitive?

Ans: Yes. `"Yes"` and `"yes"` are different. Use `toLowerCase()` before the switch for case-insensitive matching.

Q310. When to use switch and when if-else?

Ans: Use **switch** when comparing one variable with many fixed values (menu, day, grade). Use **if-else** for ranges and complex/boolean conditions.

Q311. What is the new switch expression (Java 14+)?

Ans: It uses arrows `->`, needs no `break` (no fall-through), and can return a value.

```
String name = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    case 6, 7 -> "Weekend";
    default -> "Other";
};
```

Q312. What is the `yield` keyword in switch?

Ans: In a switch expression with a block `{ }`, `yield` returns the value of that case.

Q313. Can switch be nested?

Ans: Yes, a switch can be inside a case of another switch.

Q314. Can we use multiple values in one case?

Ans: In the old style, stack the labels (`case 1: case 2: ...`). In the new style, use commas (`case 1, 2 ->`).

Q315. Can a switch use relational conditions like `case x > 5`?

Ans: No in the classic switch. Use if-else. (Pattern matching with guards in recent Java versions allows more.)

Q316. Write a simple calculator using switch.

```
char op = '+';
int a = 10, b = 5;
switch (op) {
    case '+': System.out.println(a + b); break;
    case '-': System.out.println(a - b); break;
    case '*': System.out.println(a * b); break;
    case '/': System.out.println(a / b); break;
    default:  System.out.println("Invalid operator");
}
```

Topic 18: Break & Continue (Pages 69–73)

Loop Termination, Iteration Skip

Q317. What does the break statement do?

Ans: It **terminates** the nearest loop or switch immediately, and control moves to the statement after it.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) break;
    System.out.println(i);   // prints 1 2 3 4
}
```

Q318. What does the continue statement do?

Ans: It **skips the rest of the current iteration** and goes to the next iteration of the loop.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.println(i);   // prints 1 2 4 5
}
```

Q319. What is the difference between break and continue?

Ans: **break** exits the loop completely. **continue** skips only the current iteration and the loop keeps running.

Q320. Can break be used outside a loop or switch?

Ans: No. Compile error: "break outside switch or loop". (A labeled break can be used on a labeled block.)

Q321. Can continue be used in a switch?

Ans: Only if the switch is inside a loop. `continue` then applies to the loop.

Q322. What happens with break in nested loops?

Ans: It exits only the **innermost** loop where it is written. The outer loop continues.

Q323. What are labeled break and labeled continue?

Ans: They let you break/continue an **outer** loop from inside an inner loop by naming the loop with a label.

```
outer:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (j == 2) break outer;   // exits both loops
        System.out.println(i + " " + j);
    }
}
```

Q324. What is the danger of using continue in a while loop?

Ans: If the increment is written **after** the continue, it is skipped and the loop can become infinite. Put the update before continue.

```
int i = 0;
while (i < 5) {
    i++;                // update BEFORE continue
    if (i == 3) continue;
    System.out.println(i);
}
```

Q325. What is the difference between break, continue and return?

- **break** – exits the loop/switch.
- **continue** – skips to the next iteration.
- **return** – exits the whole **method** (optionally giving a value).

Q326. What is the difference between break and System.exit(0)?

Ans: break leaves a loop only. `System.exit(0)` stops the entire JVM/program.

Q327. Write a program to stop a loop when a number divisible by 7 is found.

```
for (int i = 1; i <= 100; i++) {
    if (i % 7 == 0) {
        System.out.println("Found: " + i);
        break;
    }
}
```

Q328. Write a program to print numbers 1 to 10 but skip the even numbers.

```
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) continue;
    System.out.println(i);
}
```

Q329. Is any code allowed after break or continue in the same block?

Ans: No. Code after them in the same block is unreachable and gives a compile error ("unreachable statement").

Q330. Where are break and continue used in real projects?

Ans: Stop searching once the item is found, skip invalid records while processing a list, and exit loops on a stop signal.

Topic 19: Java Arrays (Pages 74–79)

Declaration, Initialization, Traversal

Q331. What is an array?

Ans: A container object that stores a **fixed number of elements of the same data type** in contiguous memory. Elements are accessed with an index starting from **0**.

Q332. How do you declare an array in Java?

```
int[] arr;        // preferred
int arr2[];        // also valid
String[] names;
```

Q333. How do you initialise an array?

```
int[] a = new int[5];             // size 5, all values 0
int[] b = {10, 20, 30};            // with values
int[] c = new int[]{1, 2, 3};      // with values
a[0] = 100;                        // assign by index
```

Q334. What is the default value of an array element?

Ans: Same as default values of the type: **0** for numbers, **false** for boolean, **'\u0000'** for char, **null** for objects/Strings.

Q335. How do you find the length of an array?

Ans: Use the `length` **property** (no brackets): `arr.length`. For a String it is a method: `str.length()`.

Q336. What happens if you access an index out of bounds?

Ans: **ArrayIndexOutOfBoundsException** at runtime. Valid indexes are 0 to length-1.

Q337. What happens if you create an array with a negative size?

Ans: **NegativeArraySizeException** at runtime.

Q338. Can an array hold different data types?

Ans: No, one array holds one type. But an `Object[]` array can hold different types of objects (everything is an Object).

Q339. Is the array an object? Where is it stored?

Ans: Yes. Arrays are objects, stored in the **heap**; the reference variable is on the stack.

Q340. Can we change the size of an array after creating it?

Ans: No, array size is fixed. To grow, create a new bigger array and copy (`Arrays.copyOf`), or use `ArrayList`.

Q341. What is the difference between an array and an ArrayList?

Point	Array	ArrayList
Size	Fixed	Dynamic (grows)
Data type	Primitives and objects	Objects only (wrappers)
Length	arr.length	list.size()
Methods	None	add, remove, contains, etc.
Speed	Slightly faster	Slightly slower
Package	Built-in language feature	java.util

Q342. How do you traverse (loop through) an array?

```
int[] arr = {10, 20, 30};
for (int i = 0; i < arr.length; i++) {   // classic for
    System.out.println(arr[i]);
}
for (int x : arr) {                      // for-each
    System.out.println(x);
}
```

Q343. How do you print an array?

Ans: Printing directly shows an address-like text (`[I@1b6d3586`). Use `Arrays.toString(arr)` (1D) or `Arrays.deepToString(arr)` (2D).

Q344. What is the purpose of `Arrays.sort()`?

Ans: It sorts the array in ascending order (`java.util.Arrays`). For objects you can pass a `Comparator` for custom order.

```
int[] a = {5, 2, 9, 1};
Arrays.sort(a);
System.out.println(Arrays.toString(a));   // [1, 2, 5, 9]
```

Q345. How would you reverse an array in Java?

Ans: Swap elements from both ends moving towards the middle.

```
int[] arr = {1, 2, 3, 4, 5};
for (int i = 0, j = arr.length - 1; i < j; i++, j--) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
System.out.println(Arrays.toString(arr));   // [5, 4, 3, 2, 1]
```

Q346. Find the largest element in an array.

```
int[] arr = {4, 9, 2, 7};
int max = arr[0];
for (int n : arr) {
    if (n > max) max = n;
}
System.out.println(max);   // 9
```

Q347. Find the second largest element in an array.

Ans: Track two values in a single pass.

```
int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
for (int n : arr) {
    if (n > first)       { second = first; first = n; }
    else if (n > second && n != first) { second = n; }
}
```

Q348. Find the sum and average of array elements.

```
int sum = 0;
for (int n : arr) sum += n;
double avg = (double) sum / arr.length;
```

Q349. How do you search an element in an array?

Ans: Linear search with a loop, or `Arrays.binarySearch(arr, key)` — which works **only on a sorted array**.

Q350. How do you copy an array?

- `Arrays.copyOf(arr, newLength)`
- `System.arraycopy(src, srcPos, dest, destPos, length)`
- `arr.clone()`

Note: `int[] b = a;` does **not** copy; both refer to the same array.

Q351. How do you compare two arrays?

Ans: `arr1 == arr2` compares references. Use `Arrays.equals(arr1, arr2)` to compare contents.

Q352. What is a two-dimensional (2D) array?

Ans: An array of arrays (rows and columns), like a table or matrix.

```
int[][] m = { {1, 2, 3}, {4, 5, 6} };
System.out.println(m[1][2]);     // 6
for (int i = 0; i < m.length; i++) {
    for (int j = 0; j < m[i].length; j++) {
        System.out.print(m[i][j] + " ");
    }
    System.out.println();
}
```

Q353. What is a jagged array?

Ans: A 2D array where each row can have a different length: `int[][] a = new int[3][]; a[0] = new int[2]; a[1] = new int[4];`

Q354. How to remove duplicates or find duplicate elements in an array?

Ans: Use a `HashSet`: if `set.add(x)` returns false, it is a duplicate. Or compare using nested loops.

Q355. How do you fill an array with one value?

Ans: `Arrays.fill(arr, 7);`

Q356. What is ArrayStoreException?

Ans: Thrown when you store a wrong type into an array whose actual type is different, e.g. `Object[] o = new String[2]; o[0] = 10;`.

Q357. Can we pass an array to a method? Does the method change the original?

Ans: Yes. The array reference is passed (by value), so changes to the elements inside the method **affect the original array**.

Q358. Can the size of an array be given as a long or a variable?

Ans: Size can be a variable but must be an `int` (or smaller type like short/char). A long is not allowed.

Q359. What is the output of `int[] a = new int[3]; System.out.println(a[0]);`?

Ans: **0**, the default value for int.

Topic 20: String Fundamentals (Pages 81–90)

String Pool, Immutability, == vs equals(), intern()

Q360. What is a String in Java?

Ans: A String is an **object** that represents a sequence of characters. It belongs to `java.lang.String`, is **immutable** and is a `final` class.

Q361. In how many ways can you create a String object?

Ans: Two ways:

- **String literal**: `String s1 = "Kodewala";`
- **new keyword**: `String s2 = new String("Kodewala");`

Q362. What is the String Constant Pool (SCP)?

Ans: A special area in the heap memory where String **literals** are stored. When you create a literal, Java first checks the pool; if the same text exists, it **reuses** it instead of creating a new object. This saves memory.

Q363. What is the difference between a String literal and `new String()`?

Ans: A literal goes to the pool and is reused. `new String()` **always creates a new object in the heap** (plus the literal in the pool if not already there). So literals save memory.

```
String a = "Java";
String b = "Java";
String c = new String("Java");
System.out.println(a == b);       // true  (same pool object)
System.out.println(a == c);       // false (c is a separate heap object)
System.out.println(a.equals(c));  // true  (same content)
```

Q364. How many objects are created by `String s = new String("Kodewala");`?

Ans: **Two** objects (if "Kodewala" is not already in the pool): one in the String Constant Pool (the literal) and one in the heap (because of new). If it already exists in the pool, then only **one** new heap object is created. (Very common interview question.)

Q365. How many objects are created if `new String("Kodewala")` is written twice?

Ans: 3 objects in total: 1 in the pool (shared) + 2 separate heap objects.

Q366. What does immutable mean? Why is String immutable?

Ans: Immutable = **cannot be changed after creation**. Any change creates a **new object**. Reasons:

- **String pool safety** – many references share one object; changing it would affect all.
- **Security** – used for passwords, file paths, DB URLs, class names.
- **Thread safety** – can be shared between threads without synchronisation.
- **Hashcode caching** – hash is calculated once, making HashMap keys fast.

Q367. Show immutability with an example.

```
String s = "Hello";
s.concat(" World");          // result is thrown away
System.out.println(s);       // Hello (unchanged)

s = s.concat(" World");      // new object assigned to s
System.out.println(s);       // Hello World
```

Q368. How does immutability affect performance?

Ans: Every change (like concatenation in a loop) creates a new object, so it wastes memory and time. For many modifications use `StringBuilder` / `StringBuffer`.

Q369. What is the difference between == and equals() for Strings?

- `==` compares **references** (are both variables pointing to the same object?).
- `equals()` compares the **content** (characters).

Always use `equals()` to compare String values.

Q370. What happens if you compare two different String objects with ==?

Ans: The result is **false**, even if the text is the same, because they are different objects in memory.

Q371. What is the output? `String a = new String("hi"); String b = new String("hi"); a == b; a.equals(b);`

Ans: `a == b` → **false** (two heap objects). `a.equals(b)` → **true** (same content).

Q372. What does the intern() method do?

Ans: It returns the **pool version** of the String. If the pool already has that text, it returns that reference; otherwise it adds the string to the pool and returns it.

```
String s1 = new String("Java");
String s2 = s1.intern();
String s3 = "Java";
System.out.println(s2 == s3);   // true
System.out.println(s1 == s3);   // false
```

Q373. What is the significance of intern()?

Ans: It saves memory when you have many duplicate strings created at runtime (e.g., read from files/DB) and also allows fast `==` comparison after interning.

Q374. What is the output of "Java" == "Ja" + "va" ?

Ans: **true**. Both sides are compile-time constants, so the compiler joins them to "Java" in the pool. But if one part is a variable (not final), the concatenation happens at runtime and creates a new object, so == is false.

Q375. How do you concatenate Strings in Java? What is the most efficient way?

Ans: Ways: + operator, `concat()`, `StringBuilder.append()`, `StringBuffer.append()`, `String.join()`, `String.format()`. For simple joins + is fine. In **loops**, `StringBuilder` is the most efficient.

Kodewala Java Interview

Q376. Performance: String vs StringBuilder?

Ans: Using + in a loop creates a new String every time (slow, more garbage). StringBuilder modifies one object in place, so it is much faster.

Q377. What is the difference between String, StringBuilder and StringBuffer?

Point	String	StringBuffer	StringBuilder
Mutable	No (immutable)	Yes	Yes
Thread-safe	Yes (immutable)	Yes (synchronized)	No
Speed	Slow for changes	Slower than Builder	Fastest
Since	Java 1.0	Java 1.0	Java 1.5
Use when	Fixed text	Multi-threaded changes	Single-thread changes

Q378. Why is the String class final?

Ans: So that no one can extend it and change its immutable behaviour. This protects security and the String pool.

Q379. Why is String widely used as a HashMap key?

Ans: Because it is immutable, its hashCode never changes and it is cached, so lookups are fast and safe.

Q380. Why is char[] preferred over String for storing passwords?

Ans: A String stays in the pool until garbage collected and cannot be erased. A char array can be overwritten (filled with blanks) after use, so the password does not remain in memory.

Q381. How do you reverse a String in Java?

```
String s = "Kodewala";
String rev = new StringBuilder(s).reverse().toString();

// Or with a loop
String r = "";
for (int i = s.length() - 1; i >= 0; i--) {
    r += s.charAt(i);
}
```

Q382. How do you check whether a String is a palindrome?

Ans: Reverse it and compare with the original using `equals()`.

```
String s = "madam";
String rev = new StringBuilder(s).reverse().toString();
System.out.println(s.equals(rev) ? "Palindrome" : "Not palindrome");
```

Q383. Count the occurrences of a character in a String.

```
String s = "banana";
int count = 0;
for (int i = 0; i < s.length(); i++) {
    if (s.charAt(i) == 'a') count++;
}
System.out.println(count);   // 3
```

Q384. Count vowels in a String.

```
int v = 0;
for (char ch : s.toLowerCase().toCharArray()) {
    if ("aeiou".indexOf(ch) != -1) v++;
}
```

Q385. How do you check if two Strings are anagrams?

Ans: Convert both to char arrays, sort them and compare.

```
char[] a = s1.toCharArray();
char[] b = s2.toCharArray();
Arrays.sort(a);
Arrays.sort(b);
System.out.println(Arrays.equals(a, b));
```

Q386. How do you compare Strings ignoring case? How to check alphabetical order?

Ans: `equalsIgnoreCase()` ignores case. `compareTo()` compares dictionary order: returns 0 if equal, negative if first is smaller, positive if bigger.

Q387. How do you avoid NullPointerException while comparing a String?

Ans: Put the literal first: `"admin".equals(userName)` instead of `userName.equals("admin")`.

Q388. Where is the String pool located in memory?

Ans: In the **heap** (since Java 7). Before Java 7 it was in PermGen.

Q389. Can we create a String without a literal or new? (Other ways)

Ans: Yes: `String.valueOf(10)`, `new String(charArray)`, `obj.toString()`, `sb.toString()`, `String.format()`, text blocks `""" ... """` (Java 15+).

Q390. Can we extend (inherit) the String class?

Ans: No, because it is `final`.

Q391. Is String thread-safe?

Ans: Yes. Because it is immutable, its content cannot change, so it can be safely shared across threads.

Topic 21: String Methods (Pages 91–96)

length(), charAt(), substring(), replace(), split(), concat()

Q392. What does `length()` do?

Ans: Returns the number of characters in the String. `"Java".length()` → 4.

Q393. What does `charAt(index)` do?

Ans: Returns the character at the given index (starting from 0). `"Java".charAt(2)` → 'v'. An invalid index throws **StringIndexOutOfBoundsException**.

Q394. What does `substring()` do? Explain both forms.

Ans: Returns a part of the String.

- `substring(begin)` – from begin to the end.
- `substring(begin, end)` – from begin to **end-1** (end is exclusive).

```
String s = "Kodewala";
s.substring(4);      // "wala"
s.substring(0, 4);   // "Kode"
```

Kodewala_Java Interview

Q395. What does `indexOf()` and `lastIndexOf()` do?

Ans: `indexOf()` gives the index of the **first** occurrence of a char/String; `lastIndexOf()` gives the **last** one. Both return **-1** if not found.

```
"banana".indexOf('a');       // 1
"banana".lastIndexOf('a');   // 5
"banana".indexOf('z');       // -1
```

Q396. What is the difference between `replace()` and `replaceAll()`?

Ans: `replace()` replaces plain characters/text (all occurrences). `replaceAll()` takes a **regular expression**. Also `replaceFirst()` replaces only the first match.

```
"a b c".replace(" ", "_");        // a_b_c
"a1b2".replaceAll("[0-9]", "");   // ab
```

Q397. How does `split()` work?

Ans: Splits a String into a **String array** based on a delimiter (regular expression).

```
String csv = "Java,Python,C";
String[] parts = csv.split(",");
// parts = ["Java", "Python", "C"]
```

Q398. How do you split on special characters like `.` or `|`?

Ans: They are regex characters, so escape them: `split("\\.")` and `split("\\|")`. Otherwise results are wrong. To split by spaces of any number: `split("\\s+")`.

Q399. What does the limit parameter in `split(regex, limit)` do? Are trailing empty strings kept?

Ans: limit controls the maximum number of parts. By default (limit 0), **trailing empty strings are removed**; use a negative limit (e.g., -1) to keep them.

Q400. What does `concat()` do? Difference from +?

Ans: Joins another String at the end and returns a new String. `concat()` accepts only a String (and throws NPE for null), while `+` accepts any type and treats null as "null".

Q401. What do `toUpperCase()` and `toLowerCase()` do?

Ans: Return a new String with all letters in upper/lower case. The original is unchanged.

Q402. What does `trim()` do? What is `strip()`?

Ans: `trim()` removes leading and trailing whitespace. `strip()` (Java 11) does the same but is Unicode-aware. `stripLeading()` and `stripTrailing()` remove one side.

Q403. What is the difference between `isEmpty()` and `isBlank()`?

Ans: `isEmpty()` is true if length is 0. `isBlank()` (Java 11) is true if the String is empty **or contains only spaces**.

Q404. What do `contains()`, `startsWith()` and `endsWith()` do?

Ans: Return a boolean: `contains("ab")` – text is present anywhere; `startsWith("ab")` and `endsWith("ab")` check the beginning/end.

Q405. What is the difference between `equals()`, `equalsIgnoreCase()` and `compareTo()`?

- `equals()` – content check, case sensitive, boolean.
- `equalsIgnoreCase()` – content check ignoring case.
- `compareTo()` – returns an int (0 = equal, <0 smaller, >0 greater) for sorting.

Kodewala Java Interview

Q406. How do you convert a String to a char array and back?

Ans: `char[] ch = s.toCharArray();` and `String s2 = new String(ch);` (or `String.valueOf(ch)`).

Q407. How do you convert other types to String and String to numbers?

Ans: To String: `String.valueOf(10)` or `10 + ""`. From String: `Integer.parseInt("10")`, `Double.parseDouble("3.5")`.

Q408. What does `String.join()` do?

Ans: Joins many Strings with a delimiter: `String.join("-", "a", "b", "c")` → "a-b-c".

Q409. What does `String.format()` do?

Ans: Creates a formatted String using format specifiers like `%d` (int), `%s` (String), `%.2f` (decimal with 2 places).

```
String.format("Name: %s, Age: %d", "Ravi", 25);
```

Q410. What does `repeat()` do?

Ans: (Java 11) Repeats the String n times: `"ab".repeat(3)` → "ababab".

Q411. What does `matches()` do?

Ans: Checks whether the whole String matches a regular expression: `"12345".matches("[0-9]+")` → true.

Q412. How do you replace all spaces with underscores in a String?

Ans: `s.replace(" ", "_")`

Q413. Does any String method change the original String?

Ans: No. All String methods return a **new String**. You must store the result: `s = s.toUpperCase();`.

Q414. What exception occurs in `substring()` and `charAt()` for wrong indexes?

Ans: **StringIndexOutOfBoundsException**.

Q415. How do you find the first non-repeating character in a String?

Ans: Count each character (array of 26/256 or LinkedHashMap) and return the first character with count 1.

```
String s = "swiss";
for (char c : s.toCharArray()) {
    if (s.indexOf(c) == s.lastIndexOf(c)) {
        System.out.println(c);   // w
        break;
    }
}
```

Q416. How do you remove all white spaces from a String?

Ans: `s.replaceAll("\\s", "")`

Q417. How do you split a sentence into words and count them?

Ans: `String[] words = sentence.trim().split("\\s+"); words.length`

Kodewala Java Interview

Topic 22: StringBuffer (Pages 97–100)

Mutable Strings, Common Methods

Q418. What is StringBuffer?

Ans: A class (`java.lang`) that represents a **mutable** (changeable) sequence of characters. It modifies the same object, and is **thread-safe** because its methods are synchronized.

Q419. What are the common methods of StringBuffer?

- `append()` – add text at the end.
- `insert(index, text)` – insert at a position.
- `delete(start, end)` / `deleteCharAt(i)` – remove text.
- `replace(start, end, text)` – replace a range.
- `reverse()` – reverse the content.
- `capacity()`, `length()`, `charAt()`, `setCharAt()`, `setLength()`, `toString()`.

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");        // Hello World
sb.insert(5, ",");          // Hello, World
sb.replace(0, 5, "Hi");     // Hi, World
sb.delete(2, 3);            // Hi World
sb.reverse();               // dlroW iH
System.out.println(sb);
```

Q420. What is the difference between String and StringBuffer?

Ans: String is **immutable** (every change makes a new object). StringBuffer is **mutable** (changes happen in the same object), so it is more efficient for frequent modifications. StringBuffer is also thread-safe.

Q421. When would you use StringBuffer instead of String or StringBuilder?

Ans: Use it when text is modified frequently **and** the object is shared by multiple threads (needs thread safety). For single-thread code, StringBuilder is better.

Q422. What are the performance considerations when using StringBuffer?

Ans: Better than String for many modifications (no new objects). But it is **slower than StringBuilder**, because every method is synchronized (lock overhead).

Q423. Why is StringBuffer thread-safe?

Ans: Its methods are `synchronized`, so only one thread can use them at a time on the same object.

Q424. What is the default capacity of a StringBuffer?

Ans: **16** characters for `new StringBuffer()`. If you give a String, capacity = **length of that String + 16**. Example: `new StringBuffer("abc")` → capacity 19.

Q425. What happens if the capacity of a StringBuffer exceeds its limit?

Ans: It automatically grows. The new capacity formula is **(old capacity × 2) + 2**. A new internal array is created and the old content is copied.

```
StringBuffer sb = new StringBuffer();
System.out.println(sb.capacity());   // 16
sb.append("12345678901234567");      // 17 chars > 16
System.out.println(sb.capacity());   // (16 * 2) + 2 = 34
```

Kodewala Java Interview

Q426. Difference between `length()` and `capacity()`?

Ans: `length()` is the number of characters currently stored. `capacity()` is the amount of space available before it needs to grow.

Q427. How can we set the capacity of a StringBuffer?

Ans: Pass it in the constructor `new StringBuffer(100)`, or call `ensureCapacity(n)`. `trimToSize()` reduces the capacity to the current length.

Q428. Why is the StringBuffer class declared as final?

Ans: To prevent subclassing, so no one can override its synchronized methods and break the thread-safe behaviour. It also keeps the class design secure and consistent.

Q429. Does StringBuffer override equals()?

Ans: No. `equals()` behaves like `==` (reference comparison). To compare content, convert to String using `toString().equals(...)`.

Q430. How do you convert between String and StringBuffer?

Ans: String → StringBuffer: `new StringBuffer(str)`. StringBuffer → String: `sb.toString()`.

Q431. Can we use method chaining with StringBuffer?

Ans: Yes. Methods return the same object, so you can chain:
`sb.append("a").append("b").reverse();`

Q432. Is StringBuffer immutable like String? Is it stored in the String pool?

Ans: No to both. It is mutable and is created as a normal object in the heap (never in the pool).

Q433. How many objects are created? `StringBuffer sb = new StringBuffer("Kode"); sb.append("wala");`

Ans: One StringBuffer object in the heap (plus the literal "Kode" and "wala" in the pool). Append modifies the same StringBuffer, no new one is created.

Q434. Is StringBuffer used a lot today?

Ans: It is a legacy (Java 1.0) class. In modern code StringBuilder is used for most cases; StringBuffer only when thread safety is truly needed.

Topic 23: StringBuilder (Pages 101–103)

Mutable Strings, Dynamic Text Building

Q435. What is StringBuilder?

Ans: A mutable sequence of characters, introduced in Java 1.5. It has the **same methods as StringBuffer** but is **not synchronized**, so it is faster.

Q436. Explain the difference between StringBuilder and StringBuffer.

Point	StringBuilder	StringBuffer
Thread-safe	No	Yes (synchronized)
Performance	Faster	Slower
Since	Java 1.5	Java 1.0
Use in	Single-threaded code	Multi-threaded code

Kodewala Java Interview

Q437. Why is StringBuilder preferred when multiple string manipulations are required?

Ans: It changes the same object (no new objects like String) and has no synchronisation overhead (unlike StringBuffer). So it is the fastest and most memory-efficient choice.

Q438. In a multi-threaded environment, would you use StringBuilder? Why or why not?

Ans: **No**, if the same StringBuilder object is shared between threads, because it is not thread-safe and the data can get corrupted. Use StringBuffer (or synchronise manually). It is fine when each thread has its own StringBuilder.

Q439. When would you use StringBuilder in a real-time project?

- Building **dynamic SQL queries** based on filters.
- Creating JSON/XML/CSV text or log messages.
- Joining many strings inside loops.
- Building reports or HTML content.

Q440. What is the default capacity of StringBuilder?

Ans: **16** characters. With an initial String it is `string length + 16`.

Q441. What happens when the length exceeds the current capacity?

Ans: It grows automatically. New capacity = **(old capacity × 2) + 2**. A new bigger array is created and old content is copied.

Q442. How can you change the capacity of a StringBuilder?

Ans: Use the constructor `new StringBuilder(500)`, `ensureCapacity(int)` to increase, and `trimToSize()` to reduce it to the current length. Setting a good initial capacity avoids repeated resizing.

Q443. Show the commonly used StringBuilder methods.

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" Rocks");        // Java Rocks
sb.insert(4, "!");          // Java! Rocks
sb.deleteCharAt(4);         // Java Rocks
sb.replace(0, 4, "Kotlin"); // Kotlin Rocks
sb.reverse();               // skcoR niltoK
String result = sb.toString();
```

Q444. How do you clear a StringBuilder?

Ans: `sb.setLength(0);` (or `sb.delete(0, sb.length())`).

Q445. Write a program to build a dynamic SQL query from user filters.

Ans: Start with `WHERE 1=1` so that every filter can be added simply with `AND`.

Kodewala Java Interview

```
String dept = "IT";
int salary = 50000;
String joinDate = "2020-01-01";

StringBuilder sql = new StringBuilder("SELECT * FROM employees WHERE 1=1");
if (dept != null) {
    sql.append(" AND department = '").append(dept).append("'");
}
if (salary > 0) {
    sql.append(" AND salary > ").append(salary);
}
if (joinDate != null) {
    sql.append(" AND joining_date >= '").append(joinDate).append("'");
}
System.out.println(sql);
```

Q446. Reverse a String using StringBuilder.

```
String rev = new StringBuilder("Kodewala").reverse().toString();   // alawedoK
```

Q447. Why is concatenation with + in a loop slow compared to StringBuilder?

Ans: Each + creates a new String object and copies the old content, so n iterations create n objects ($O(n^2)$ copying). StringBuilder appends into one buffer.

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append(i).append(",");
}
```

Q448. Does StringBuilder override equals() and hashCode()?

Ans: No. `equals()` compares references only. Compare content with `sb1.toString().equals(sb2.toString())` (or `compareTo` in Java 11+).

Q449. Is the StringBuilder stored in the String pool? Is the class final?

Ans: It is created in the heap like any object (not in the pool). Yes, it is a `final` class.

Q450. Does the Java compiler use StringBuilder internally?

Ans: Yes. For + concatenations, the compiler converts them to StringBuilder (older versions) or uses `invokedynamic` (Java 9+). But inside loops it still creates a new builder each time, so write StringBuilder manually.

Q451. Quick summary: String vs StringBuilder vs StringBuffer – which to choose?

- Text never changes → **String**.
- Many changes, single thread → **StringBuilder**.
- Many changes, shared between threads → **StringBuffer**.

— End of Document. All the best for your interview! —

Kodewala Java Interview